Práctica: ampliación de tools y gestión del historial en EBIS-CLI

Contexto

EBIS-CLI es un agente de línea de comandos que recibe un objetivo en lenguaje natural, consulta un modelo de OpenAI y puede ejecutar funciones Python locales cuando el modelo solicita una tool. La versión inicial ya incluye la interfaz, la configuración del cliente, tres herramientas para trabajar con archivos y el loop que ejecuta las herramientas y recopila sus resultados.

En esta práctica ampliarás el agente con dos tools sencillas y completarás su memoria de conversación. El punto de partida contiene dos problemas deliberados: faltan las nuevas herramientas y el modelo recibe solo el objetivo actual, aunque la aplicación sí conserva una lista de historial. Además, el historial puede crecer sin límite.

Completa únicamente los dos bloques señalados en el código:

- `ebis_cli/tools.py`: crear `count_words` y `calculate_total` y conectarlas al modelo.
- `ebis_cli/agent.py`: contar las palabras del historial, compactarlo cuando supere el límite y corregir el envío de la conversación al modelo.

Objetivos

Al completar la práctica serás capaz de:

- Implementar una función Python que se pueda invocar como tool.
- Describir sus argumentos mediante un esquema JSON para OpenAI.
- Registrar una tool para que el agente pueda ejecutarla.
- Seguir el ciclo de una llamada a una tool con la Responses API.
- Mantener el historial entre objetivos del usuario.
- Contar las palabras del historial y resumirlo cuando supere un umbral.
- Preparar el resumen según el objetivo actual.

Material inicial

La carpeta del proyecto es:

```
ebis-cli-proyect/
├── ebis_cli/
│   ├── agent.py
│   ├── app.py
│   ├── context.py
│   ├── style.tcss
│   └── tools.py
├── .env.example
├── .gitignore
├── enunciado.md
├── pyproject.toml
└── README.md
```

`app.py`, `context.py` y la interfaz se entregan preparados. No necesitas modificar la interfaz ni la gestión de rutas. `tools.py` ya contiene `list_files`, `read_file` y `write_file`. `agent.py` ya contiene el loop de interacción con OpenAI y la ejecución de las llamadas a tools.

Preparación del entorno

Se utiliza `uv` para preparar el entorno e iniciar la aplicación. Solo hacen falta dos comandos:

- `uv sync` crea el entorno virtual local e instala las dependencias del proyecto.
- `uv run ...` ejecuta un comando usando ese entorno.

Desde la carpeta `ebis-cli-proyect`, ejecuta:

```
uv sync
```

Crea `.env` a partir del ejemplo:

```
cp .env.example .env
```

Abre `.env` y sustituye `OPENAI_API_KEY` por tu clave. `OPENAI_MODEL` ya tiene un valor de ejemplo; puedes cambiarlo por un modelo al que tengas acceso. No compartas ni subas `.env` al repositorio.

Al terminar los ejercicios, iniciarás la aplicación así:

```
uv run python -m ebis_cli.app
```

También puedes iniciar el agente desde otra carpeta de trabajo. Usa `--project` para indicar dónde está el proyecto EBIS-CLI:

```
cd /home/alumno/proyectos/mi-workspace
cp /home/alumno/ebis-cli-proyect/.env.example .env
# Edita .env y añade tu OPENAI_API_KEY.
uv run --project /home/alumno/ebis-cli-proyect python -m ebis_cli.app
```

`--project` le indica a `uv` dónde encontrar `pyproject.toml` y el entorno con las dependencias del agente. No cambia la carpeta de trabajo actual: el proceso se inicia desde `mi-workspace`, por lo que las tools usan esa carpeta para leer y escribir archivos. En este proyecto, `context.py` también carga `.env` desde la carpeta actual; por eso el workspace necesita su propio `.env`. La ruta del proyecto se usa para ejecutar el código y resolver sus dependencias, mientras que la carpeta desde la que lanzas el comando determina el workspace. La documentación de `uv` describe esta diferencia entre `--project` y cambiar el directorio de trabajo.

Lo mínimo necesario de la SDK de OpenAI

El proyecto usa la SDK oficial de Python y la Responses API. `context.py` carga las variables de `.env`; por eso `AsyncOpenAI()` puede leer `OPENAI_API_KEY` del entorno. Como el agente es asíncrono, cada petición se espera con `await`.

La llamada principal ya está en `agent.py`:

```
response = await self.client.responses.create(
    model=self.model,
    instructions=self.instructions,
    input=run_history,
    tools=TOOLS,
)
```

Los cuatro datos que debes reconocer son:

- `model`: el modelo elegido en la configuración.
- `instructions`: las instrucciones generales del agente.
- `input`: el contenido que se envía al modelo. Puede ser un texto, o una lista de mensajes y elementos de una conversación.
- `tools`: los esquemas que describen las herramientas disponibles.

Una llamada a `responses.create` no recibe automáticamente los mensajes anteriores guardados en Python. Para dar continuidad a la conversación, el programa debe incluirlos en `input` en cada petición. En esta práctica, la lista preparada en `run_history` contiene el historial anterior, el objetivo actual y, durante el loop, las llamadas y resultados de tools. La línea del proyecto que pasa `input=goal` ignora esa lista.

La respuesta tiene dos partes que ya utiliza el código:

- `response.output_text` reúne el texto de respuesta del modelo.

- `response.output` contiene los elementos generados. Si el modelo solicita una función, uno de ellos tendrá `type == "function_call"`, el nombre en `name`, los argumentos JSON en `arguments` y el identificador de llamada en `call_id`.

El loop ya interpreta los argumentos con `json.loads`, llama a `dispatch_tool` y añade a la conversación un elemento `function_call_output` asociado al `call_id`. Al enviar de nuevo la conversación, el modelo puede leer el resultado y decidir qué hacer después. En la documentación oficial puedes consultar el ejemplo de estado de conversación y el ciclo de function calling.

Archivos a completar

```
ebis_cli/tools.py  -> TODO 1: dos herramientas, sus esquemas y el dispatcher
ebis_cli/agent.py  -> TODO 2: historial, límite de palabras y envío a la API
```

TODO 1: crear count_words y calculate_total

Abre `ebis_cli/tools.py`. El agente conoce por ahora solo las tres tools de archivos. Añadir una función Python no basta para que el modelo pueda usarla: el esquema la anuncia a la API y el dispatcher conecta esa llamada con el código local. Mantén el mismo nombre en las tres piezas.

1. Implementar las funciones Python

Añade las funciones junto a las herramientas existentes, antes de la lista `TOOLS`.

`count_words` recibe `text: str` y cuenta las palabras separadas por espacios en blanco. En Python, `text.split()` ya separa correctamente palabras aunque haya espacios repetidos o saltos de línea. Devuelve el resultado como texto, por ejemplo `"Número de palabras: 4"`.

`calculate_total` recibe `unit_price: float` y `quantity: int`. Calcula `unit_price * quantity` y devuelve el total como texto con dos decimales, por ejemplo `"Total: 37.50"`.

2. Declarar los esquemas

Añade una entrada por tool a `TOOLS`. La estructura de cada entrada debe seguir el formato que ya utilizan las tres herramientas existentes:

- `type` vale `"function"`.
- `name` coincide exactamente con el nombre de la función Python.
- `description` explica qué hace la tool.
- `parameters` declara los argumentos con tipos JSON Schema.
- `required` incluye todos los argumentos.
- `additionalProperties` vale `False`.

- `strict` vale `True`.

Para `count_words`, declara un objeto con una propiedad `text` de tipo `string` y descríbela como el texto del que se contarán las palabras.

Para `calculate_total`, declara un objeto con `unit_price` de tipo `number` y `quantity` de tipo `integer`. Describe el primero como el precio unitario y el segundo como el número de unidades. Ambos son obligatorios.

Usa como referencia las entradas de `TOOLS` que ya existen y adapta sus propiedades a los argumentos de cada herramienta. Para `calculate_total`, el esquema debe reflejar los tipos `number` e `integer` indicados arriba.

3. Registrar las funciones

Al final de `tools.py`, localiza el diccionario `functions` dentro de `dispatch_tool`. Conserva las tres entradas existentes y añade una entrada por cada tool nueva, usando el nombre de la función como clave y la función como valor. El dispatcher ya se ocupa de pasar los argumentos y convertir los errores en un resultado legible.

Cuando las tres piezas estén conectadas, `/tools` debe mostrar también `count_words` y `calculate_total`.

TODO 2: reparar y limitar el historial

Abre `ebis_cli/agent.py`. Lee primero el método `run`: copia `self.history` en `run_history`, añade el objetivo actual y va agregando a esa lista las respuestas del modelo y los resultados de las tools. Al final del run, los mensajes nuevos se guardan en `self.history`.

La aplicación conserva mensajes en Python, pero eso no garantiza que el modelo los reciba. Además, cada objetivo y cada intercambio con una tool puede aumentar el historial. El agente tiene dos problemas relacionados con la memoria:

1. La llamada de OpenAI pasa `input=goal`, así que el modelo recibe únicamente el objetivo actual. No recibe el historial ni los resultados de tools que el loop acaba de añadir.
2. `self.history` puede crecer con cada objetivo y acumular más palabras de las que queremos enviar en una petición.

Trabaja sobre `MAX_HISTORY_WORDS`, cuyo valor inicial es 300 palabras.

1. Crear una función que cuente palabras

Define el método `count_history_words(self, history)` dentro de `Agent`, antes de `run`. Debe recorrer los elementos de la lista y sumar las palabras de su campo `content`, usando `len(text.split())` para contar.

Los mensajes de usuario son diccionarios y los elementos de `response.output` son objetos de la SDK. El método debe poder leer `content` de ambos formatos. Si `content` es una lista de bloques, cuenta el texto de cada bloque.

Ignora los elementos que no tengan texto en `content`, como los resultados de tools que usan el campo `output`. El método debe devolver un entero.

2. Compactar cuando se supera el umbral

Si se envía el historial completo en cada paso, puede crecer hasta resultar difícil de manejar. La compactación debe reducirlo solo cuando exceda el umbral, conservando lo útil para el objetivo que acaba de llegar.

Define el método asíncrono `compact_history(self, goal)` dentro de `Agent`, antes de `run`, siguiendo este flujo:

1. Llama a `count_history_words(self.history)`.
2. Si el resultado es menor o igual a `MAX_HISTORY_WORDS`, termina sin cambiar `self.history` ni llamar a OpenAI.
3. Si el resultado es mayor, solicita al modelo un resumen breve del historial. En `instructions`, pídele que conserve solo los hechos, decisiones y resultados que podrían ayudar con el objetivo actual, sin inventar información y sin resolver el objetivo.
4. En la entrada de esa petición incluye `self.history` y un mensaje que indique el valor de `goal` como objetivo actual. Así el resumen se adapta a la tarea que se va a iniciar.
5. Sustituye `self.history` por un único mensaje de asistente cuyo `content` sea el resumen obtenido mediante `response.output_text`.

La petición de compactación también usa `await self.client.responses.create(...)`. No incluyas `tools=TOOLS` en esa petición: solo necesitas generar texto de resumen.

3. Corregir la llamada del loop

Al pedir una tool, el agente ejecuta la función y añade su llamada y su resultado a `run_history`. Sin embargo, esos elementos no sirven para el siguiente paso si la petición a OpenAI solo incluye el texto del objetivo. El mismo problema ocurre entre objetivos: la lista `self.history` se guarda, pero no se envía.

La API de Responses no lee automáticamente una lista de Python que el programa mantiene en memoria. Para que el modelo pueda continuar la conversación, esa lista debe formar parte de `input`. Inspecciona qué mensajes reúne `run_history` y ajusta la llamada principal para que el modelo reciba esa conversación en cada paso.

También debes ejecutar la compactación antes de preparar `run_history`: al principio de `run`, llama a `await self.compact_history(goal)`, copia el historial resultante y añade el objetivo nuevo.

No borres los `run_history.extend(response.output)` ni el bloque que agrega `function_call_output`: son necesarios para el siguiente paso del loop.

1. Arranque y comprobación de las tools

Desde `ebis-cli-proyect`, inicia la aplicación:

```
uv run python -m ebis_cli.app
```

Escribe `/tools` y confirma que se muestran las cinco herramientas. Prueba estas instrucciones, indicando expresamente que el agente debe usar la herramienta:

```
Usa count_words para contar las palabras de esta frase: "EBIS aprende a crear agentes útiles".
```

El resultado esperado es 6 palabras.

```
Usa calculate_total con precio unitario 12.50 y cantidad 3.
```

El resultado esperado es `37.50`.

Si la tool no se ejecuta, revisa en este orden: el nombre en el esquema, la entrada en `dispatch_tool` y los nombres de los argumentos. Los argumentos del esquema deben coincidir con los parámetros de la función Python.

Referencias

- Documentación de la SDK oficial de Python
- Gestionar el estado de una conversación con Responses
- Function calling con la Responses API
- Documentación de uv